

Q1. Employee Performance & Payroll Dashboard

The **Employee Performance & Payroll Dashboard** is a Java AWT-based GUI application developed to manage employee details, performance ratings, and payroll calculations. The application accepts employee ID, employee name, department, designation, basic salary, working days, and performance rating. It uses AWT components such as Frame, Label, TextField, Choice, Checkbox, Button, and TextArea.

The GUI is arranged using **GridBagLayout**, which provides flexible positioning of components. The application uses **ActionListener** to handle events generated by the Generate Report and Reset buttons. Input validation is performed to ensure that all required fields are entered and that salary and working days contain valid values.

The payroll calculation includes adjusted salary based on working days, HRA, performance bonus, gross salary, tax, and net salary. The performance bonus is calculated according to the selected rating such as Excellent, Good, or Average. After clicking **Generate Report**, the complete employee payroll details and calculated salary are displayed in the TextArea. The **Reset** button clears all entered data and restores the form to its initial state.

Thus, the application provides a simple and efficient method for maintaining employee information and generating accurate payroll reports.

```

BorderLayoutDemo.java x
1      import java.awt.*;
2      import java.awt.event.*;
3
4      class BorderLayoutDemo extends Frame implements ActionListener {
5
6          TextField id, name, salary, days; 5 usages
7          Choice department, designation; 8 usages
8          Checkbox excellent, good, average; 3 usages
9          CheckboxGroup performance; 6 usages
10         TextArea report; 8 usages
11         Button generate, reset; 4 usages
12
13     BorderLayoutDemo() { 1 usage
14         setTitle("Employee Performance & Payroll Dashboard");
15         setSize( width: 700, height: 600);
16         setLayout(new GridBagLayout());
17
18         GridBagConstraints g = new GridBagConstraints();
19         g.insets = new Insets( top: 5, left: 5, bottom: 5, right: 5);
20         g.fill = GridBagConstraints.HORIZONTAL;
21
22         id = new TextField();
23         name = new TextField();
24         salary = new TextField();
25         days = new TextField();
26
27         department = new Choice();
28         department.add("CSE");
29         department.add("IT");
30         department.add("ECE");
31         department.add("HR");
32
33         designation = new Choice();
34         designation.add("Developer");
35         designation.add("Tester");
36         designation.add("Manager");
37         designation.add("Analyst");
38
39         performance = new CheckboxGroup();
40         excellent = new Checkbox( label: "Excellent", performance, state: true);
41         good = new Checkbox( label: "Good", performance, state: false);
42         average = new Checkbox( label: "Average", performance, state: false);
43
44         report = new TextArea( rows: 12, columns: 45);
45         report.setEditable(false);
46
47         generate = new Button( label: "Generate Report");
48         reset = new Button( label: "Reset");
49
50         addComponent(new Label( text: "Employee ID"), id, g, row: 0);
51         addComponent(new Label( text: "Employee Name"), name, g, row: 1);
52         addComponent(new Label( text: "Department"), department, g, row: 2);
53         addComponent(new Label( text: "Designation"), designation, g, row: 3);
54         addComponent(new Label( text: "Basic Salary"), salary, g, row: 4);
55         addComponent(new Label( text: "Working Days"), days, g, row: 5);
56
57         g.gridx = 0;

```

```

63         p.add(good);
64         p.add(average);
65
66         g.gridx = 1;
67         add(p, g);
68
69         g.gridx = 0;
70         g.gridy = 7;
71         g.gridwidth = 2;
72
73         Panel buttons = new Panel();
74         buttons.add(generate);
75         buttons.add(reset);
76         add(buttons, g);
77
78         g.gridy = 8;
79         add(report, g);
80
81         generate.addActionListener(this);
82         reset.addActionListener(this);
83
84         addWindowListener(new WindowAdapter() {
85             public void windowClosing(WindowEvent e) {
86                 dispose();
87             }
88         });
89
90         setVisible(true);

```

```

96         g.gridy = row;
97         g.gridwidth = 1;
98         add(label, g);
99
100        g.gridx = 1;
101        add(field, g);
102    }
103
104    public void actionPerformed(ActionEvent e) {
105
106        if (e.getSource() == generate) {
107
108            try {
109                if (id.getText().trim().isEmpty() ||
110                    name.getText().trim().isEmpty() ||
111                    salary.getText().trim().isEmpty() ||
112                    days.getText().trim().isEmpty()) {
113
114                    report.setText("Please fill all required fields.");
115                    return;
116                }
117
118                double basic = Double.parseDouble(salary.getText());
119                int workingDays = Integer.parseInt(days.getText());
120
121                if (basic <= 0 || workingDays <= 0 || workingDays > 26) {
122                    report.setText("Enter valid salary and working days (1-26).");
123                    return;
124                }

```

```

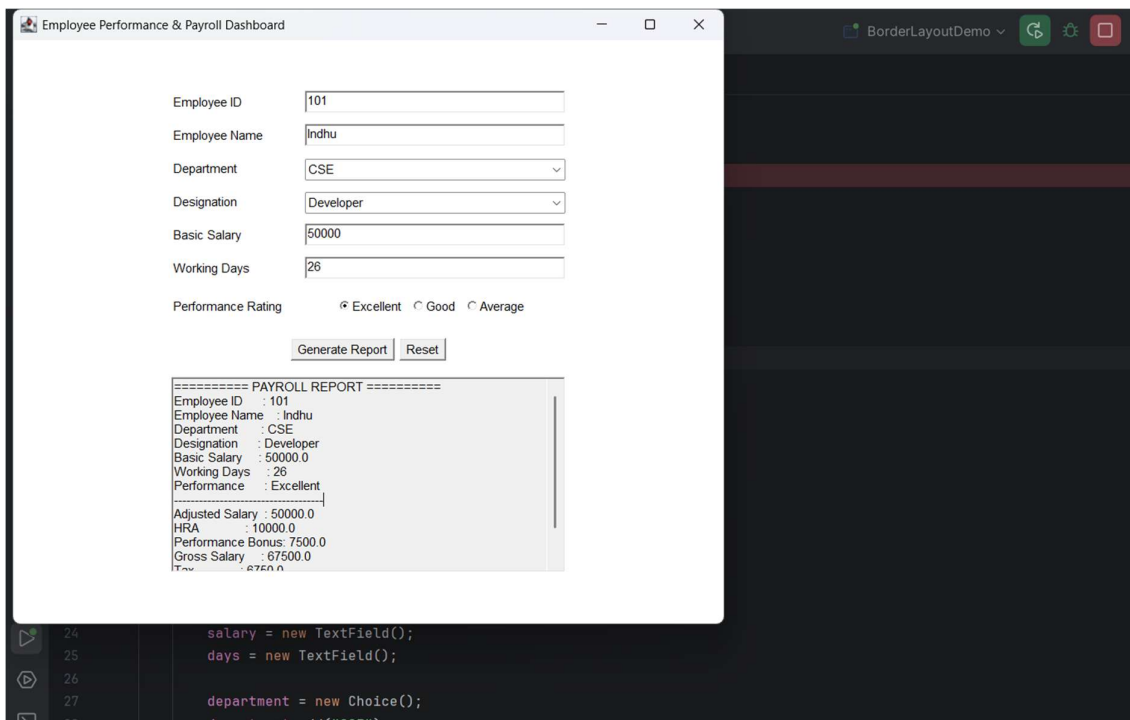
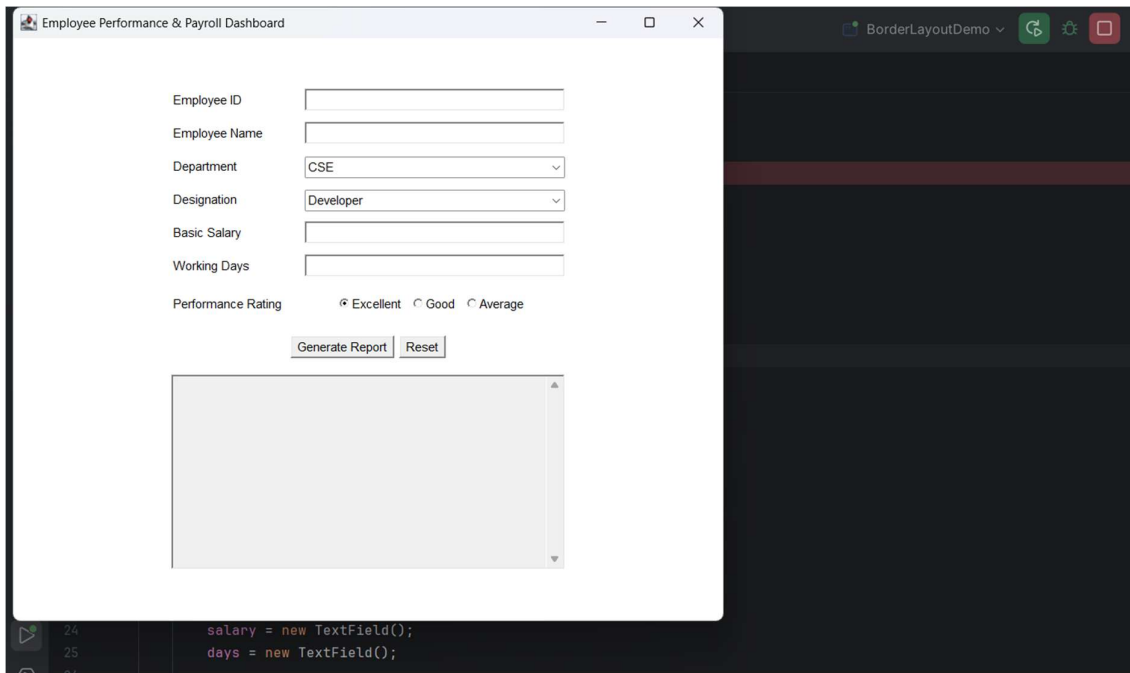
141         double gross = adjustedSalary + hra + bonus;
142         double tax = gross * 0.10;
143         double net = gross - tax;
144
145         report.setText(
146             "===== PAYROLL REPORT =====\n" +
147                 "Employee ID      : " + id.getText() + "\n" +
148                 "Employee Name   : " + name.getText() + "\n" +
149                 "Department     : " + department.getSelectedItem() + "\n" +
150                 "Designation    : " + designation.getSelectedItem() + "\n" +
151                 "Basic Salary   : " + basic + "\n" +
152                 "Working Days   : " + workingDays + "\n" +
153                 "Performance    : " + rating + "\n" +
154                 "-----\n" +
155                 "Adjusted Salary : " + adjustedSalary + "\n" +
156                 "HRA             : " + hra + "\n" +
157                 "Performance Bonus: " + bonus + "\n" +
158                 "Gross Salary    : " + gross + "\n" +
159                 "Tax             : " + tax + "\n" +
160                 "Net Salary      : " + net + "\n" +
161                 "=====");
162     };
163
164     } catch (NumberFormatException ex) {
165         report.setText("Please enter valid numeric values.");
166     }
167 }
168

```

```

162     );
163
164     } catch (NumberFormatException ex) {
165         report.setText("Please enter valid numeric values.");
166     }
167 }
168
169 if (e.getSource() == reset) {
170     id.setText("");
171     name.setText("");
172     salary.setText("");
173     days.setText("");
174     department.select(pos: 0);
175     designation.select(pos: 0);
176     performance.setSelectedCheckbox(excellent);
177     report.setText("");
178 }
179 }
180
181 public static void main(String[] args) {
182     new BorderLayoutDemo();
183 }
184

```



Q2. Online Food Ordering and Billing System

The **Online Food Ordering and Billing System** is a Java AWT-based GUI application used to select food items and automatically generate a bill. The system consists of three screens: **Customer Details, Food Selection, and Bill Generation**. Customer name and table number are entered on the first screen. The second screen allows the user to select a food category, food item, and quantity.

The application uses AWT components such as Frame, Panel, Label, TextField, Choice, Button, and TextArea. **CardLayout** is used to switch between the different screens, while **ActionListener** handles button events such as Next, Previous, Add Item, Generate Bill, and Clear.

When a food item is added, the system calculates the item amount using **Price × Quantity**. Multiple food items can be added to the order. The subtotal is calculated from all selected items, and **5% GST** is added to obtain the final bill amount. Validation is provided to ensure that the quantity entered is a valid positive number and that customer details are provided.

The final screen displays the customer details, ordered items, subtotal, GST, and final payable amount. The Clear button resets the application. Hence, the system provides a simple, accurate, and user-friendly solution for food ordering and billing.

```

BorderLayoutDemo.java x
1  import java.awt.*;
2  import java.awt.event.*;
3
4  public class BorderLayoutDemo extends Frame implements ActionListener {
5
6      CardLayout card; 6 usages
7      Panel main; 9 usages
8
9      TextField customer, table, qty; 5 usages
10     Choice category, food; 8 usages
11     TextArea orderArea, billArea; 9 usages
12
13     Button next, previous, add, generate, clear; 4 usages
14
15     double total = 0; 6 usages
16
17     BorderLayoutDemo() { 1 usage
18         setTitle("OnLine Food Ordering and Billing System");
19         setSize( width: 650, height: 500);
20
21         card = new CardLayout();
22         main = new Panel(card);
23
24         // Customer Screen
25         Panel p1 = new Panel(new GridLayout( rows: 4, cols: 2, hgap: 10, vgap: 10));
26
27         p1.add(new Label( text: "Customer Name"));
28         customer = new TextField();
29         p1.add(customer);
30

```

```

31         p1.add(new Label( text: "Table Number"));
32         table = new TextField();
33         p1.add(table);
34
35         next = new Button( label: "Next");
36         previous = new Button( label: "Previous");
37         p1.add(next);
38         p1.add(previous);
39
40         main.add(p1, constraints: "customer");
41
42         // Food Selection Screen
43         Panel p2 = new Panel(new GridLayout( rows: 6, cols: 2, hgap: 10, vgap: 10));
44
45         p2.add(new Label( text: "Food Category"));
46         category = new Choice();
47         category.add("Pizza");
48         category.add("Burger");
49         category.add("Drinks");
50         p2.add(category);
51
52         p2.add(new Label( text: "Food Item"));
53         food = new Choice();
54         food.add("Margherita Pizza - 200");
55         food.add("Cheese Burger - 150");
56         food.add("Cold Drink - 80");
57         p2.add(food);
58

```

```

60     qty = new TextField();
61     p2.add(qty);
62
63     add = new Button( label: "Add Item");
64     generate = new Button( label: "Generate Bill");
65     previous = new Button( label: "Previous");
66     clear = new Button( label: "Clear");
67
68     p2.add(add);
69     p2.add(generate);
70     p2.add(previous);
71     p2.add(clear);
72
73     orderArea = new TextArea();
74     p2.add(orderArea);
75     p2.add(new Label( text: ""));
76
77     main.add(p2, constraints: "food");
78
79     // Bill Screen
80     Panel p3 = new Panel(new BorderLayout());
81
82     billArea = new TextArea();
83     billArea.setEditable(false);
84     p3.add(billArea, BorderLayout.CENTER);
85
86     Panel bp = new Panel();
87     previous = new Button( label: "Previous");

```

```

111         setVisible(true);
112     }
113
114
115     void updateFood() { 1 usage
116         food.removeAll();
117
118         if (category.getSelectedItem().equals("Pizza")) {
119             food.add("Margherita Pizza - 200");
120             food.add("Veg Pizza - 250");
121         } else if (category.getSelectedItem().equals("Burger")) {
122             food.add("Cheese Burger - 150");
123             food.add("Veg Burger - 120");
124         } else {
125             food.add("Cold Drink - 80");
126             food.add("Fresh Juice - 100");
127         }
128     }
129
130     public void actionPerformed(ActionEvent e) {
131
132         if (e.getSource() == next) {
133             if (customer.getText().trim().isEmpty() ||
134                 table.getText().trim().isEmpty()) {
135                 orderArea.setText("Enter customer name and table number.");
136                 return;
137             }
138             card.show(main, name: "food");
139         }

```



```

4      public class BorderLayoutDemo extends Frame implements ActionListener {
30          public void actionPerformed(ActionEvent e) {
41              else if (e.getSource() == previous) {
42                  card.previous(main);
43              }
44
45              else if (e.getSource() == add) {
46                  try {
47                      int q = Integer.parseInt(qty.getText());
48
49                      if (q <= 0) {
50                          orderArea.setText("Enter valid quantity.");
51                          return;
52                      }
53
54                      String item = food.getSelectedItem();
55                      double price = getPrice(item);
56                      double amount = price * q;
57
58                      total += amount;
59
60                      orderArea.append(item + " x " + q +
61                                     " = Rs." + amount + "\n");
62
63                      qty.setText("");
64
65                  } catch (Exception ex) {
66                      orderArea.setText("Enter a valid quantity.");
67                  }
68              }

```

```

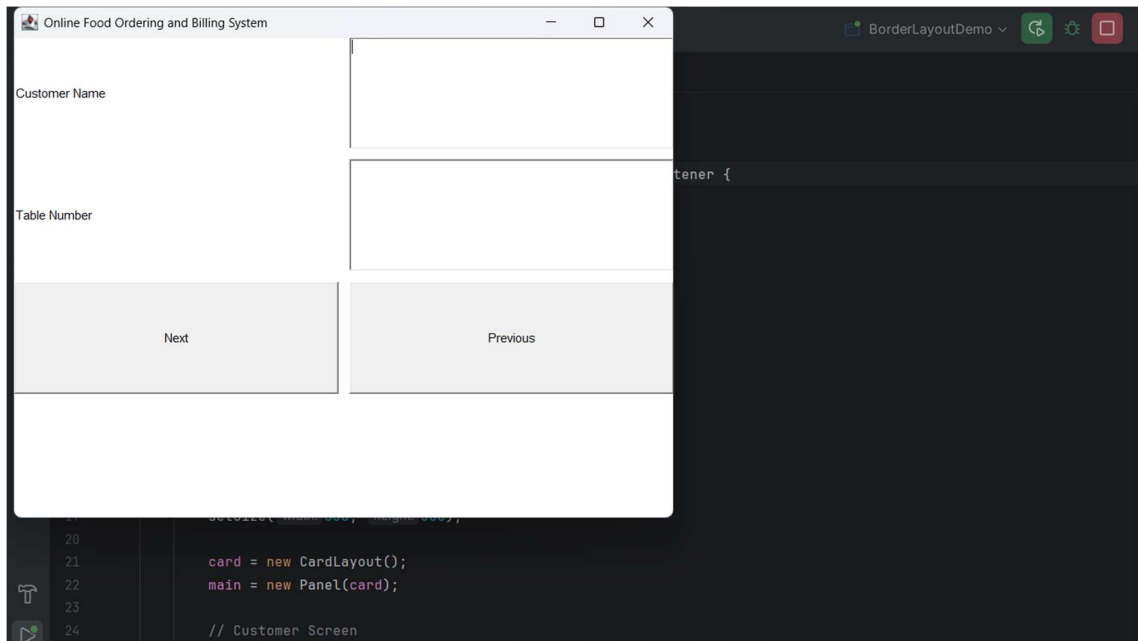
171          if (total == 0) {
172              orderArea.setText("Please add at least one item.");
173              return;
174          }
175
176          double gst = total * 0.05;
177          double finalAmount = total + gst;
178
179          billArea.setText(
180              "===== FINAL BILL =====\n\n" +
181              "Customer Name : " + customer.getText() + "\n" +
182              "Table Number : " + table.getText() + "\n\n" +
183              "----- ITEMS ----- \n" +
184              orderArea.getText() +
185              "\n-----\n" +
186              "Subtotal      : Rs." + total + "\n" +
187              "GST (5%)       : Rs." + gst + "\n" +
188              "Final Amount  : Rs." + finalAmount + "\n" +
189              "===== \n" +
190              "Thank You! Visit Again!"
191          );
192
193          card.show(main, name: "bill");
194      }
195
196      else if (e.getSource() == clear) {
197          customer.setText("");
198          table.setText("");

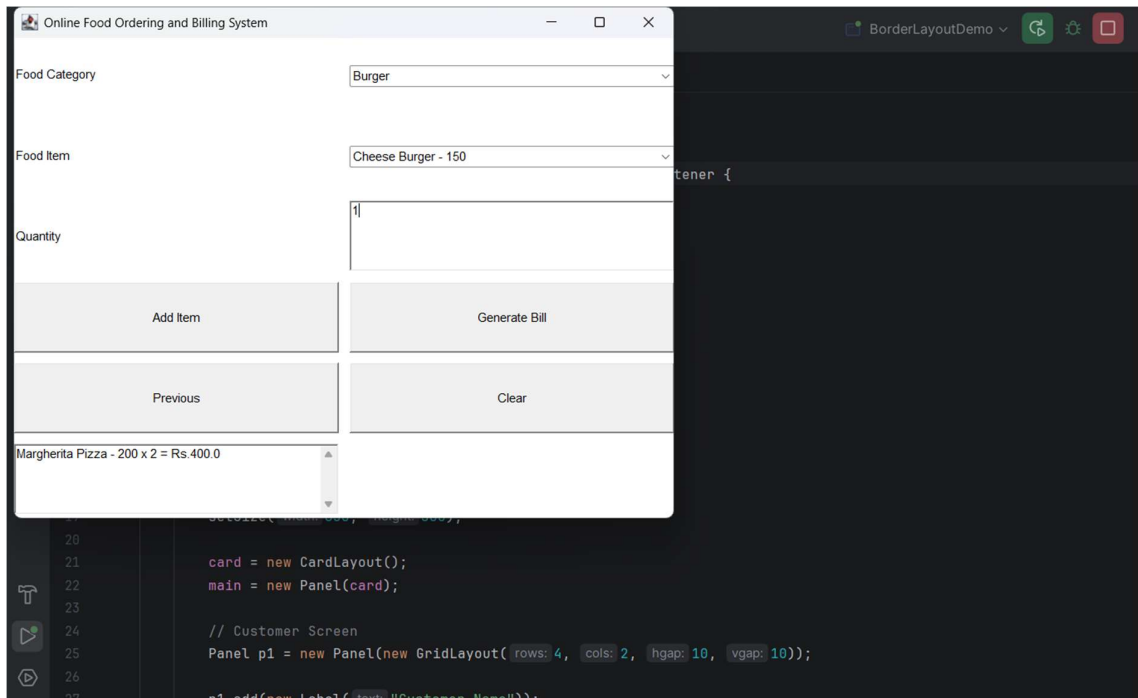
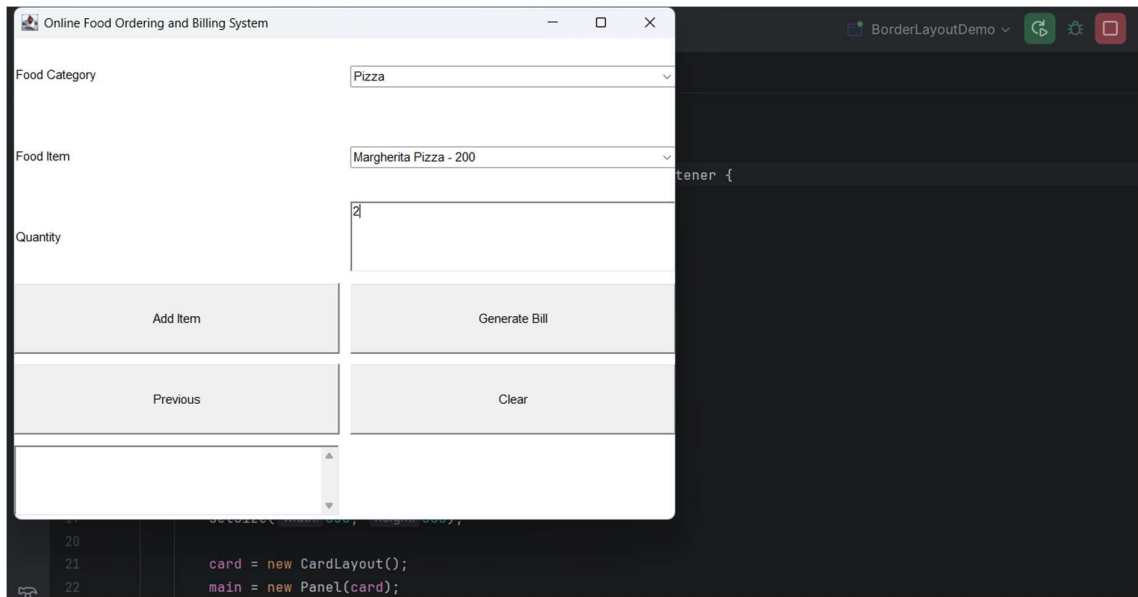
```

```

196         else if (e.getSource() == clear) {
197             customer.setText("");
198             table.setText("");
199             qty.setText("");
200             orderArea.setText("");
201             billArea.setText("");
202             total = 0;
203             card.show(main, name: "customer");
204         }
205     }
206
207     @
208     double getPrice(String item) { 1 usage
209         if (item.contains("Margherita"))
210             return 200;
211         if (item.contains("Veg Pizza"))
212             return 250;
213         if (item.contains("Cheese Burger"))
214             return 150;
215         if (item.contains("Veg Burger"))
216             return 120;
217         if (item.contains("Cold Drink"))
218             return 80;
219         return 100;
220     }
221
222     public static void main(String[] args) {
223         new BorderLayoutDemo();
224     }

```





Online Food Ordering and Billing System

===== FINAL BILL =====

Customer Name : Indhu
Table Number : 10

----- ITEMS -----
Margherita Pizza - 200 x 2 = Rs.400.0

Subtotal : Rs.400.0
GST (5%) : Rs.20.0
Final Amount : Rs.420.0

=====

Thank You! Visit Again!

Previous

Clear

BorderLayoutDemo

tener {

20

21

22

23

24

25

26

27

28

card = new CardLayout();

main = new Panel(card);

// Customer Screen

Panel p1 = new Panel(new GridLayout(rows: 4, cols: 2, hgap: 10, vgap: 10));

p1.add(new Label(text: "Customer Name"));

customer = new TextField();